AMRITA VISHWA VIDYAPEETHAM

Department of Artificial Intelligence

AmazeBot

An AI-Powered Document, Character-Voice & Molecule Discovery Assistant

A Project Report Submitted in Partial Fulfilment of the Requirements for
B.Tech in Artificial Intelligence and Data Science
Course: Introduction to Material Informatics (Course Code: 23MAT204)

Submitted by	**G Venugopalan (CB.AI.U4AID25115)**
	Selva Vignesh (CB.AI.U4AID25149)
	Mahalakshmi R (CB.AI.U4AID25167)
Under the guidance of	**Dr. Suman Dutta**
Head of the Department	**Dr. Soman K P**

Amrita Vishwa Vidyapeetham
2026

CERTIFICATE

This is to certify that the project report entitled **"AmazeBot: An AI-Powered Document, Character-Voice & Molecule Discovery Assistant"** is a bonafide record of the work carried out by:

- G Venugopalan — CB.AI.U4AID25115
- Selva Vignesh — CB.AI.U4AID25149
- Mahalakshmi R — CB.AI.U4AID25167

in partial fulfilment of the requirements for the course **Introduction to Material Informatics (23MAT204)**, B.Tech in Artificial Intelligence and Data Science, at Amrita Vishwa Vidyapeetham, during the academic year 2026, under the supervision of the undersigned.

Dr. Suman Dutta
Faculty Guide, Course Faculty — Introduction to Material Informatics

Dr. Soman K P
Head of the Department, Department of Artificial Intelligence

Place: _____ Date: _____

DECLARATION

We hereby declare that the project report entitled **"AmazeBot: An AI-Powered Document, Character-Voice & Molecule Discovery Assistant"**, submitted for the course **Introduction to Material Informatics (23MAT204)**, B.Tech in Artificial Intelligence and Data Science, Amrita Vishwa Vidyapeetham, is a record of original work carried out by us under the guidance of **Dr. Suman Dutta**. We further declare that this report, or any part of it, has not been submitted elsewhere for the award of any degree, diploma, or other similar title.

Place: _____ Date: _____

G Venugopalan
CB.AI.U4AID25115 _____

Selva Vignesh
CB.AI.U4AID25149 _____

Mahalakshmi R
CB.AI.U4AID25167 _____

TABLE OF CONTENTS

Abstract

AmazeBot is an AI-powered assistant that combines three tools in one application: a document Q&A assistant that answers questions grounded in an uploaded paper or report and generates flashcards and quizzes from it; a character-chat feature that lets users talk to custom AI personas using cloned, streamed voice and live hands-free voice calls; and a molecule-discovery tool that turns a plain-language description of a desired material into candidate molecules with interactive 3D structures. The system runs either on the cloud (Gemini API) or fully offline on a local machine with no internet or recurring cost. For molecule discovery, it integrates live with a teammate's independent AI model over a real API, and automatically falls back to locally generated results if that service fails, so the tool stays honest and usable at all times. The project demonstrates a working, integrated implementation of the Conversational-AI and Molecular-Discovery-AI components required by the course.

1. Introduction

1.1 Background

Technical material — research papers, datasheets, lab reports — is time-consuming to read in full when a reader only needs an answer to one specific question. Separately, inverse molecular design (asking a generative model to propose molecules with target properties) is a genuinely useful technique, but it typically exists only as research code, inaccessible to anyone without a specialist's setup. Finally, most student-built AI assistants are text-only, which does not reflect how people naturally think through a problem — by talking it through, interactively, with follow-up.

1.2 Why This Problem Is Worth Solving

The course brief requires an integrated system spanning three connected architecture blocks: a **Conversational AI (CAI)** component, a **Molecular-Discovery AI (MD-by-AI)** component, and a **Report Generation** component, working together rather than as isolated demonstrations. AmazeBot is our team's implementation of the CAI block, built to connect live — over a real network API — with a teammate's independently developed MD-by-AI block.

1.3 What AmazeBot Is

AmazeBot is a single web application containing three connected tools:

1. **Document Chat** — upload a document, ask grounded questions, generate flashcards and quizzes, and use a Design-AI assistant mode.
2. **Character Chat + Voice** — create custom AI personas with a cloned voice, interact by text or by a live, hands-free voice call.
3. **Molecule Discovery** — describe a target material in plain language and receive candidate molecules with computed formulas and live 3D structures.

It runs in two modes: a **cloud mode** powered by the Google Gemini API, and a **fully offline local mode** powered by a local LLM and local voice models, requiring no API key and no internet connection once set up.

2. Problem Statement

#	Problem	Why It Is Hard	Why We Are Solving It
1	**Information overload.** Reading a full document to answer one question does not scale.	Answers must stay faithful to the source, across both cloud and offline models, without an expensive retrieval pipeline.	Directly reduces study/research time; this is the CAI block required by the course.
2	**Inaccessible specialist tools.** Inverse molecular design exists only as research code.	Requires integrating live with an independent, still-imperfect external model, and staying honest when it fails.	Without an interface, the underlying MD-by-AI research has no usable front end.
3	**Shallow interaction.** Text-only assistants do not match natural, spoken interaction.	Real-time voice cloning, streaming synthesis, and hands-free turn-taking are genuine audio-engineering problems.	Turns a query tool into something people will actually use conversationally.

3. Objectives

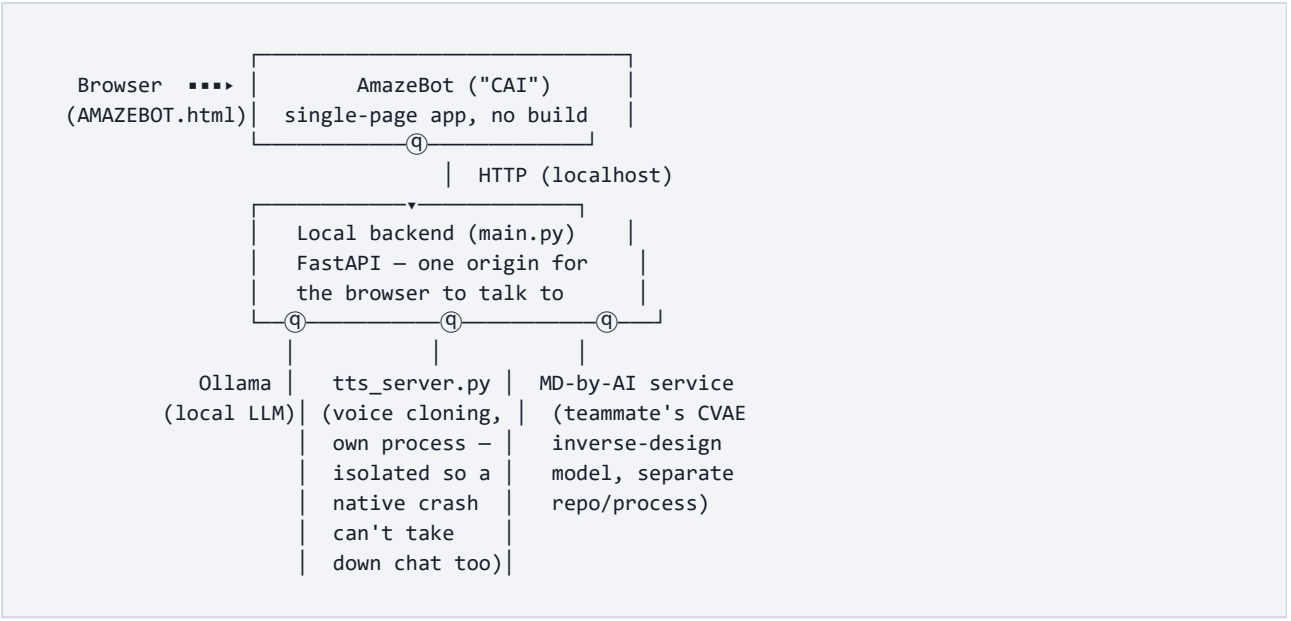
- Answer questions from a document using the document's own text, not the model's general knowledge.
- Give the assistant a natural interaction layer: custom personas with cloned, streamed voice and live hands-free voice calls.

- Integrate end-to-end with a teammate's independent MD-by-AI model over a live API — a real cross-project integration, not a stub.
- Support full offline operation, with zero recurring API cost and zero internet dependency once set up.

4. Literature Review

Paper	Publisher / Year	Technology Aided	Pros	Gap / Cons
Auto-Encoding Variational Bayes (Kingma & Welling)	ICLR, 2014	Variational Autoencoders (VAE)	Learns a smooth, continuous latent space	Not chemistry-specific; needs substantial data
Automatic Chemical Design via Continuous Molecule Representation (Gómez-Bombarelli et al.)	ACS Central Science, 2018	Conditional VAE for molecule generation from SMILES	First gradient-based molecule optimization	Needs 10,000s of molecules — teammate's model uses ~37
Retrieval-Augmented Generation (Lewis et al.)	NeurIPS, 2020	RAG — grounding LLM answers via retrieval	Reduces hallucination; scales to large corpora	Retrieval overhead deliberately not used here
Robust Speech Recognition — Whisper (Radford et al.)	OpenAI / arXiv, 2022	Weakly-supervised speech-to-text	Strong accuracy across accents/noise	Full models slow on consumer GPUs — we use faster-whisper
XTTS: Multilingual Zero-Shot TTS (Casanova et al., Coqui)	arXiv, 2024	Zero-shot voice cloning TTS	Clones a voice from seconds of audio	Naive generation is slow — motivated our streaming engine
Photodynamic Therapy for Cancer (Dolmans, Fukumura & Jain)	Nature Reviews Cancer, 2003	Domain background on PDT drug efficacy	Explains why target properties matter	Pre-dates generative molecular design

5. System Architecture



`main.py` is the single backend origin the browser talks to; it proxies to Ollama, to `tts_server.py`, and to the teammate's MD-by-AI service. Two reasons drive this design: browsers block cross-origin requests by default (CORS), so a single origin avoids that entirely; and isolating voice cloning into its own OS process means a native CUDA/cuDNN crash there cannot take the chat backend down with it.

6. Implementation

6.1 Technology Stack

Layer	Technology
Frontend	Single-file HTML, CSS, vanilla JavaScript — no build step, no framework
Frontend libraries (vendored)	marked.js, pdf.js, jszip.js, three.js, chart.js, KaTeX
Backend	Python, FastAPI, served via uvicorn
Local LLM	Ollama, running LiquidAI/lfm2.5-1.2b-instruct
Voice cloning / synthesis	XTTS-v2 (Coqui TTS), isolated in tts_server.py
Speech-to-text	faster-whisper
Cheminformatics	RDKit — 3D structure generation, molecular formulas

Cloud AI	Google Gemini API

6.2 Local Execution

Process	Port	Role
Static file server	8080	Serves AMAZEBOT.html and its libraries to the browser
main.py (FastAPI)	8000	The single backend origin the browser calls
tts_server.py	8001	Voice cloning, isolated process
Ollama	11434	Local LLM inference

6.3 Document Understanding — Full Context, Not Retrieval

Document Chat deliberately does not use vector embeddings or a retrieval step. For the document sizes this app targets (single papers or reports, not large corpora), the entire extracted document text is placed directly into the model's context alongside the user's question: text is extracted client-side, injected in full into the conversation context, and the LLM answers grounded in that context. This is a stated trade-off — simpler and more accurate than a full RAG pipeline for the scale this tool is used at, at the cost of not scaling to very large document collections.

6.4 Voice Engine

- **Cloning:** a short reference clip becomes cached XTTS-v2 conditioning latents, bounded to an LRU cache of 8 entries to protect a laptop GPU's limited VRAM.
- **Streaming playback:** a custom AudioWorklet ring-buffer player begins speaking as soon as the first audio chunk exists, with a jitter buffer absorbing generation-speed variance.
- **Voice Mode:** real-time RMS-based Voice Activity Detection drives hands-free turn-taking, interruptible mid-sentence.

6.5 Molecule Discovery Integration

A plain-language request is parsed into structured target parameters and sent over a live HTTP call to the teammate's MD-by-AI Cloud Run service. Returned SMILES strings are rendered as an interactive 3D structure using a hand-built parser and Three.js. If the live service fails or

degrades, a local RDKit-based fallback generates real candidates automatically, explicitly flagged as such.

6.6 APIs Used

API	Type	Purpose
Google Gemini API	External REST API	Cloud-mode chat, extraction, grading
AmazeBot's own backend API	Internal REST API	Frontend ↔ backend communication
Ollama's local API	Local REST API	Local-mode chat inference
MD-by-AI API	External REST API	Live molecule-candidate generation
Browser Web APIs	Native browser APIs	MediaRecorder, getUserMedia, AudioWorklet

7. Results

7.1 Deliverables Achieved

- Document Chat: Q&A, flashcards, quiz, and Design-AI mode
- Character Chat: custom personas with persistent memory
- Voice cloning (XTTS-v2) with real-time streamed playback
- Voice Mode: live hands-free call with automatic turn-taking
- Molecule Discovery: full-screen mode with saved search history
- Live integration with the teammate's MD-by-AI Cloud Run service
- Resilient local fallback when the live service fails or degrades
- Fully offline local mode (Ollama + on-device voice models)
- Public GitHub repository with a live hosted demo (GitHub Pages)

7.2 Progress & Validation

All three tools were implemented and tested end-to-end in both cloud and fully offline modes. Voice reliability issues were root-caused across four separate layers — a sample-rate mismatch, buffer-stitching fragility, an unbounded GPU-memory cache, and GPU contention between the local LLM and the voice model — and fixed one at a time, each verified with real audio-thread

capture rather than assumption. Quiz grading was made fair: copy-pasted questions submitted as answers are now detected and scored zero, and genuinely correct answers are no longer under-scored by an overly strict fallback heuristic.

7.3 Honest Findings From the Live External Integration

Direct testing of the teammate's live MD-by-AI service uncovered four real limitations in that separate system: a small training set (approximately 37 molecules) that limits generalization; a property-mapping mismatch between requested and trained properties; a hardcoded fallback pool of six generic molecules returned whenever generation falls short; and the live service returning an empty candidate list even while its own internal statistics reported successful generation. None of these are fixable from AmazeBot's side. AmazeBot's response was to build a transparent, RDKit-based local fallback that activates automatically and is always clearly labelled, so the feature remains usable and never silently presents an unreliable result as verified.

8. Challenges & Risks

- **Shared-GPU contention** between the local LLM and voice model caused audio glitches under real concurrent load.
- **Streaming audio engineering** required several rounds of re-architecture to achieve gapless, low-latency playback.
- **Dependency on an independently-evolving external service** for Molecule Discovery's result quality.
- **Live-demo risk** — results honestly fall back to local candidates rather than fail silently if the external service is unstable.

9. Next Steps

- Retrain the MD-by-AI model on a larger dataset and correct its property mapping.
- Replace the placeholder fitness score with a real, computed metric.
- Build the Report Generation block — the one component not yet built by anyone on the team.
- Continue reducing Voice Mode latency for an even more natural live-call experience.

10. Conclusion

AmazeBot delivers a working implementation of the Conversational-AI block required by the course, integrated end-to-end with a teammate's independent Molecular-Discovery-AI block over a live API. Every feature operates in both cloud and fully offline modes. The project integrated honestly with a real, imperfect, independently-evolving external model — identifying its actual limitations directly from its behaviour rather than assuming it worked, and engineering a transparent fallback rather than concealing failures. What is genuinely this team's own work is the complete application: the frontend, the backend, the voice engine, and the integration layer connecting to the MD-by-AI service. What is not this team's own work is the MD-by-AI model itself.

References

1. D. P. Kingma and M. Welling, “Auto-Encoding Variational Bayes,” *ICLR*, 2014.
2. R. Gómez-Bombarelli et al., “Automatic Chemical Design Using a Data-Driven Continuous Representation of Molecules,” *ACS Central Science*, 4(2), 2018.
3. P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” *NeurIPS*, 2020.
4. A. Radford et al., “Robust Speech Recognition via Large-Scale Weak Supervision,” OpenAI / arXiv:2212.04356, 2022.
5. E. Casanova et al., “XTTS: A Massively Multilingual Zero-Shot Text-to-Speech Model,” Coqui / arXiv:2406.04904, 2024.
6. M. J. Dolmans, D. Fukumura, and R. K. Jain, “Photodynamic Therapy for Cancer,” *Nature Reviews Cancer*, 3, 380–387, 2003.
7. RDKit: Open-Source Cheminformatics Software, rdkit.org.
8. FastAPI, PyTorch, Three.js, PDF.js — open-source libraries used in the AmazeBot implementation.